Identity-Bound Governance Under Execution Uncertainty

An Accountability Proof Block for LLM Agent Persistent Halts, with Cryptographic Implementation and Cross-Model Calibration

Marcelo Fernandez
TraslaIA
info@traslaia.com

Abstract

A correctly governed LLM agent can reach a state in which neither continuing execution nor automatically halting is admissible: the system has detected a persistent failure of its observability or drift-detection layer, but cannot itself decide who has the authority to resume, deny, or recalibrate the deployment. We call this an *identity-bound governance event*, and we formalise the mechanism that resolves it.

We introduce the *Accountability Proof Block* (APB), a two-part record consisting of a system-constructed evidence block E_s, a human-supplied decision block D_h, and a cryptographic signature σ_h binding the two to a registered principal $H_i \in \mathcal{P}$. The system can construct E_s but cannot forge σ_h (it lacks the principal's secret key); the principal can issue D_h but cannot alter E_s undetected (the signature covers both). We prove four theorems about this construction: *Governance Completeness* (every halt resolves through either the recovery loop or a signed APB), *Non-Repudiability* (no principal can deny a governance act for which a valid APB exists), *Impossibility of Anonymous Re-Authorization* (the execution layer cannot generate a valid APB without the principal's private credential), and *Finite-Time APB Construction Termination*.

We provide a real-cryptography implementation in `cryptography/ed25519`, expose the result through an Authority Resolution Function G, and validate the construction empirically. With a MockLLM stack across 10 seeds and 10,000 steps, governance completeness holds identically under two threshold policies (3,812 halt events, zero unresolved cases). Across 9 adversarial vectors against 200 freshly-signed APBs, the verifier achieves 100.0000% detection. A cross-model study of six open LLMs (`mistral`, `deepseek-r1`, `gemma`, `gpt-oss`, `qwen`, `llama3`) finds that the model-specific drift threshold T^* is stable within model ($\sigma/T^* < 2\%$ for all measurable cases) but varies across models by a factor of $1.7\times$, refuting any size-monotone hypothesis: the largest model in our sample is the one that does not drift within the 500-step window. A temperature sweep across three of those models at $T \in \{0.2, 0.4, 0.6, 0.8\}$ replicates the temperature-insensitivity conjecture for the two fast-drifting models (per-model range ≤ 5 steps) but reveals a *drift-floor effect* for the slow-drifter (T^* range of 51 steps for `gemma4`). We conclude that T^* must be *measured* per deployment — not predicted from architecture, and not assumed temperature-invariant for slow-drifting models — and that the APB is the operational vehicle by which that measured threshold yields accountable authority transfer.

1 Introduction

The deployment of LLM-based autonomous agents in environments where their actions have real-world consequences raises a question that prompt-level guards and policy engines do not answer:

when an agent's runtime governance system declares that it can no longer act safely, who is empowered to lift that halt, and what record must they leave behind?

We refer to this question as the problem of *identity-bound governance under execution uncertainty*. It is the structural complement to the empirical observation, established in prior work in this series [7], that runtime drift and partial observability generate halts that the recovery loop cannot itself resolve. Once such a halt is persistent in the formal sense (§3), the system is by design unable to issue a unilateral resumption: doing so would constitute a recalibration of its own authority, which the design constraints of [7] forbid.

The standard accountability mechanisms in deployed AI systems are not a solution to this problem. Documentation artefacts such as model cards [10] and OpenAI's system cards [11] are static; they describe behaviour at training or deployment time, not at the moment of an unexpected halt. Alignment techniques such as constitutional AI [2] bias the agent's behaviour but do not provide a non-repudiable record of the human decisions that surround a deployment. Compliance audit trails as mandated by GDPR, SOX, or PCI-DSS are typically post-hoc and not cryptographically bound to specific runtime events. None of these mechanisms provides a per-event, identity-bound, cryptographically verifiable record that the system itself cannot forge.

Contributions. This paper introduces the *Accountability Proof Block* (APB) as the formal object that fills this role, and validates it empirically. Specifically:

1. We define the Principal Set $\mathcal{P}$, the Authority Resolution Function G, and the APB structure $\text{APB} = (E_s, D_h, \sigma_h)$ formally (§3), and we make the threat model explicit: five attacker classes, with the boundary between what the APB defends against and what it does not made precise.
2. We prove four theorems (§4): *Governance Completeness* (T8.1), *Non-Repudiability* (T8.2), *Impossibility of Anonymous Re-Authorization* (T8.3), and *Finite-Time APB Construction Termination* (T8.4). Each is supported by auxiliary lemmas that decompose the argument.
3. We provide a real-cryptography implementation (§5) using ed25519 signatures over a canonical JSON encoding, with a Principal Registry, a verifier that distinguishes failure modes, and an append-only log with HMAC chaining for persistence.
4. We validate the construction empirically through four experiments (§6–§9): Governance Completeness over 10 seeds and two threshold policies; APB Integrity under nine adversarial vectors; cross-model T^* across six open LLM families; and temperature insensitivity across three of those families and four sampling temperatures.
5. We extract *Proposition 5.1* (§5.1) as an operational design recipe: given a measured T^* for a model, the persistence window ΔT that the execution layer uses to classify a halt as a governance event admits a calibrated range. We discuss what this means for deployments where T^* is not finite (one of our six models did not drift within the experimental window).

Independence from prior work. Although this paper sits in the *Agent Governance* series, its claims do not depend on the empirical results of prior papers. The formal framework, the theorems, and all four experiments are self-contained: fresh evidence is generated under identical software but different runs, and the cryptographic claims are entirely new (no cryptography appears in P0–P7).

Table 1: Accountability mechanisms relevant to LLM agent governance, characterised by whether they operate at runtime, are cryptographic, bind to a specific human identity, and are designed for the LLM-agent setting. The APB is, to our knowledge, the first construction that satisfies all four.

Mechanism	Runtime	Crypto	Identity-bound	LLM-agent specific
Model / system cards [10, 11]	—	—	—	✓
Constitutional AI [2]	✓	—	—	✓
Compliance audit trails (GDPR, SOX)	✓	—	—	—
Smart-contract logs [12]	✓	✓	✓	—
Threshold signatures [5]	—	✓	✓	—
APB (this work)	✓	✓	✓	✓

Where prior work is referenced (notably the design constraints DC.1 and DC.2 of [7]), the references are contextual rather than constitutive.

Roadmap. §2 situates the APB against existing accountability mechanisms. §3 introduces the Principal Set, the Authority Resolution Function, and the APB structure. §4 proves the four theorems. §5 describes the implementation and the threat model boundary. §6–§9 report the four validation experiments. §10 discusses limitations and integration paths. §11 concludes.

2 Related Work

We organise prior work along the dimensions that the APB engages simultaneously: *runtime*, *cryptographic*, *identity-bound*, and *LLM-agent specific*. To our knowledge no existing mechanism combines all four; Table 1 maps the landscape.

Documentation-based. Model cards [10], system cards [11], and data sheets are static artefacts authored prior to deployment. They describe what an agent is supposed to do or not do; they do not record what a specific human authorised the agent to resume after a halt at $t = t_e$.

Alignment-based. Constitutional AI [2] and RLHF approaches shape the agent's behaviour at training time. They affect *how* the agent acts but not *who* authorised a recovery decision. The APB is orthogonal: it does not constrain the agent's behaviour; it constrains the act of re-authorising the agent.

Audit-trail and compliance. Regulatory audit trails (GDPR Article 30, SOX section 404, PCI-DSS) log access events but typically lack cryptographic binding to a specific human credential at the moment of decision. Recent work on verifiable audit logs [9] brings cryptographic guarantees to logs themselves, but does not address per-event authorisation by named principals.

Smart contracts and on-chain governance. Smart-contract execution [12] and on-chain DAO governance produce records that are runtime, cryptographic, and identity-bound. They are, however, not designed for LLM-agent runtime state: they bind to wallet addresses (which are not the

same as human identities), assume a public ledger, and are not intended to operate at the latency of an in-process governance event. The APB borrows the cryptographic-binding insight while keeping the deployment local.

Threshold and multi-signature schemes. Threshold and multi-signature constructions [5, 8] address *which* principals must concur before an action is authorised. They complement the APB rather than replace it: §3 notes that an APB-typed governance event may require k-of-n signatures, in which case each cosigner contributes their own σ_h over the same canonical $(E_s \parallel D_h)$ payload.

Position. The APB is, to our knowledge, the first construction that is simultaneously runtime, cryptographic, identity-bound, and designed for LLM-agent governance. The closest analogues are smart-contract logs (which lack the LLM-agent fit) and threshold signatures (which are a sub-component, not a complete governance mechanism).

3 Formal Framework

3.1 Setting and Inherited Constraints

We assume an agent system whose execution layer maintains a runtime admission snapshot $\mathcal{A}_0$, a drift estimator $\widehat{D}(t)$ with threshold θ, and a halt mechanism that may emit `HALT` when its observability or risk-attribution gates fail. Two design constraints from prior work in the series [7] are inherited verbatim:

DC.1 (Separation of Recalibration Authority). For all t, $\mathcal{A}_0(t) = \mathcal{A}_0(0)$. The execution layer is forbidden from modifying its own admission baseline at runtime.

DC.2 (Drift Event Classification). A halt is *transient* if $\widehat{D}(\tau) \geq \theta$ for τ within a bounded window ΔT and returns below θ thereafter; it is *persistent* if $\widehat{D}(t) \geq \theta$ for all $t \in [t_0, t_0 + \Delta T]$. Transient halts route to the recovery loop; persistent halts constitute *governance events*.

DC.1 implies that, when a persistent halt occurs, the system cannot recalibrate its own $\mathcal{A}_0$ to clear the halt. DC.2 routes such halts out of the execution layer's authority. The role of this paper is to specify what is on the other side of that boundary.

3.2 Principal Set $\mathcal{P}$

Definition 3.1 (Principal Set)**.** *The* Principal Set *of a governed system is a finite set $\mathcal{P} = \{H_1, \ldots, H_n\}$ of identity-bound human principals. Each H_i has a public verification key pk_i and an associated private signing key sk_i held exclusively by H_i. The mapping $H_i \mapsto pk_i$ is recorded in a registry that the execution layer treats as read-only.*

DC.1 specialised to $\mathcal{P}$: the registry is fixed at $\mathcal{A}_0$ construction time, and any modification to it is itself a governance event of type `RECALIBRATE`. We thus obtain a self-referential modification discipline — $\mathcal{P}$ is mutable only through APBs that authorise its mutation.

Revocation. A principal may be revoked. We model revocation by recording a timestamp of revocation; APBs signed before the revocation timestamp remain verifiable (with a flag indicating the principal's later revoked status), while APBs signed after are rejected at verification time. This historical-validity discipline is enforced by the predicate is_active(H_i, t_e) used by the verifier (§5).

3.3 Authority Resolution Function G

Definition 3.2 (Authority Resolution Function)**.** $G : \mathcal{P} \times \mathcal{S} \times \mathcal{E} \to \{\texttt{RESUME}, \texttt{DENY}, \texttt{RECALIBRATE}\}$, *where $\mathcal{S}$ denotes the space of system runtime states and $\mathcal{E}$ the space of governance evidence packages. G is the governance-layer analog of the runtime authority function F of [7]: it constructs authority from human attestation over a defined evidence set, not from sensor state. The output* `RECALIBRATE` *is the only authorised path by which $\mathcal{A}_0$ may change after initialisation;* `RESUME` *permits a single resumption under stated scope;* `DENY` *permanently bars the queried action.*

The APB is the recorded artefact that witnesses an evaluation of G.

3.4 Accountability Proof Block (APB)

Definition 3.3 (APB)**.** *An* Accountability Proof Block *signed by principal H_i is a triple*

$$\text{APB} \;=\; (E_s,\; D_h,\; \sigma_h)$$

where

$$\begin{aligned}
E_s &= (\text{hash}(\mathcal{A}_0),\; \widehat{D}(t_e),\; t_e,\; \text{hash}(\text{trace}_{\leq t_e}),\; \text{cause}) \quad \textit{(System Evidence Block)}\\
D_h &= (H_i,\; \text{decision},\; \text{rationale},\; \text{scope}) \quad \textit{(Human Decision Block)}\\
\sigma_h &= \text{Sign}_{sk_i}(\,\text{canon}(E_s) \,\|\, \text{canon}(D_h)\,) \quad \textit{(Signature)}
\end{aligned}$$

canon$(\cdot)$ *denotes a canonical (sort-keyed, fixed-encoding) serialisation;* Sign *is the ed25519 signature operation.*

The construction has three structural properties enforced by its typing alone:

1. Only the execution layer can correctly produce E_s: it is the only party that holds $\mathcal{A}_0$, $\widehat{D}$, and the trace.
2. Only H_i can produce σ_h: it is the only party that holds sk_i.
3. σ_h covers both E_s and D_h: any modification to either invalidates the signature.

3.5 Threat Model

The APB is designed to defeat a specific class of attackers; we make the boundary of its protection explicit. Five actor classes are considered.

A1. External attacker. No system access. Goal: forge an APB attributed to H_i. Defence: T8.3 (§4).

A2. Compromised system process. Has runtime access to the execution layer but does not hold sk_i for any $H_i \in \mathcal{P}$. Goal: synthesise a σ_h that verifies against some registered pk_i. Defence: T8.3.

A3. Replay attacker. Holds a previously-valid APB and tries to use it in a new context. Defence: $t_e \in E_s$ plus the verifier's age check (§5).

A4. Man-in-the-middle. Intercepts an APB in transit and tries to modify either E_s or D_h before delivery. Defence: σ_h covers both blocks, so any tampering invalidates verification.

A5. Compromised principal. Has obtained sk_i for some H_i. *Out of scope.* The APB cryptographically binds decisions to keys; if a key is compromised, the binding attaches to the compromised key. Mitigation lies outside the APB protocol (multi-signature, key rotation, out-of-band review).

3.5.1 What APB Does Not Prevent

The APB is an *accountability* mechanism, not an *access-control* mechanism. A principal H_i holding a valid sk_i can sign any D_h they wish, including a malicious one, and the resulting APB will verify cryptographically. What the APB ensures, against this case, is that:

1. the malicious decision is attributable to H_i and cannot be denied (T8.2);
2. the system itself did not synthesise the decision behind H_i's back (T8.3).

Mitigations for adversarial principals (multi-signature, separation of duties, out-of-band review) are configurable on top of the APB but are not part of the APB protocol itself. We return to this distinction in the discussion (§10).

4 Theorems

We prove four theorems about the APB. The first three are properties promised in the framework section of [7]; we restate them precisely and give full proofs. The fourth (T8.4) is new: it establishes that the construction of E_s terminates in bounded time, which is required for T8.1 to be non-vacuous.

4.1 T8.1 — Governance Completeness

Theorem 4.1 (Governance Completeness)**.** *Let h be a* `HALT` *event emitted by an execution layer satisfying DC.1 and DC.2. Then exactly one of the following resolutions holds:*

(R1) h is transient under DC.2 and the recovery loop produces a `RESUME` *within its bounded attempt budget;*

(R2) h is persistent under DC.2 and a valid signed APB *is produced through an evaluation of G.*

No third resolution path exists.

Proof. By DC.2, h is exactly one of *transient* or *persistent*. Case (transient): the recovery loop is defined for transient halts ([7], P6 Recovery Loop) and terminates in at most k attempts with a `RESUME`, `HALT`, or `DENY` outcome on the augmented state. If the outcome is `RESUME`, we are in case (R1). If the outcome on the augmented state is `HALT` or `DENY`, the halt is reclassified as persistent (the recovery loop has exhausted the ways in which the halt could be transient), reducing to the persistent case below. Case (persistent): by DC.1 the execution layer cannot modify $\mathcal{A}_0$ to clear the halt; the only authorised path to a state in which execution may continue is an output of G. By Theorem 4.9 below, E_s can be constructed in finite time. By Definition 3.3, given E_s and a principal H_i with valid sk_i, an APB is produced. We are in case (R2). The two cases are exhaustive (DC.2) and disjoint (the recovery loop's output of `RESUME` excludes persistence within the same window). Therefore no third resolution exists. □

Lemma 4.2 (Resolution Path Exhaustivity)**.** *The recovery loop's exit set* {`RESUME`, `HALT`, `DENY`} *partitions the post-halt state space into transient (`RESUME`) and persistent (`HALT`∪`DENY`). The latter is exactly the domain of G.*

Proof. The recovery loop's outputs are mutually exclusive by construction (it returns at most one). Definition of transient (DC.2): the drift estimator returned below threshold within ΔT, observable by the recovery loop's own measurements. If `RESUME`, by P6 the loop has confirmed observability is restored, hence transient. If `HALT` or `DENY`, the loop has confirmed it cannot resolve, hence persistent. □

4.2 T8.2 — Non-Repudiability

Theorem 4.3 (Non-Repudiability)**.** *Let* APB $= (E_s, D_h, \sigma_h)$ *be a verifiable APB whose H_i field in D_h identifies a registered, non-revoked principal at t_e, and let σ_h verify against pk_i. Then no efficient adversary can produce a $D'_h \neq D_h$ on E_s that also verifies under pk_i, except with negligible probability.*

Proof. Verification of an APB requires that $\sigma_h = \mathrm{Sign}_{sk_i}(\mathrm{canon}(E_s) \,\|\, \mathrm{canon}(D_h))$. Suppose, for contradiction, that an adversary produces $\mathrm{APB}' = (E_s, D'_h, \sigma_h)$ with $D'_h \neq D_h$ that verifies. Because σ_h is a fixed value and the canonical serialisation is injective on dataclass content (Lemma 4.4), $\mathrm{canon}(D'_h) \neq \mathrm{canon}(D_h)$, so the message under the signature differs. A successful verification of σ_h over the new message contradicts the existential unforgeability of ed25519 under chosen-message attack [3, 4]. This event has negligible probability in the security parameter. □

Lemma 4.4 (E_s Post-Construction Integrity)**.** *The canonical encoding* canon *used in the APB construction is collision-resistant on dataclass content: distinct values of the five E_s fields produce distinct byte-strings.*

Proof. canon is JSON with sorted keys, fixed-encoding numeric representation, and UTF-8 string encoding. JSON with sorted keys is injective on dataclass content with primitive-type fields [6]. Each E_s field is a primitive (string or float), so the encoding is injective. □

Lemma 4.5 (Signature Verifiability Under Tamper)**.** *Let* APB *verify against pk_i with original E_s. For any modification of E_s to $E'_s \neq E_s$, retaining σ_h and D_h, the modified APB does not verify under pk_i.*

Proof. By Lemma 4.4, $\text{canon}(E'_s) \neq \text{canon}(E_s)$, hence the message under verification differs from the message that was signed. Verification of an ed25519 signature against a different message succeeds only with negligible probability. □

4.3 T8.3 — Impossibility of Anonymous Re-Authorization

Theorem 4.6 (Impossibility of Anonymous Re-Authorization)**.** *Let an adversary $\mathcal{A}$ have full read and write access to the execution layer's runtime state but not hold sk_i for any $H_i \in \mathcal{P}$. Then $\mathcal{A}$ cannot produce a verifying APB attributed to any registered principal, except with negligible probability.*

Proof. A verifying APB requires σ_h such that $\text{Verify}_{pk_i}(\sigma_h, \text{canon}(E_s) \,\|\, \text{canon}(D_h)) = \top$ for some registered pk_i. The adversary may construct E_s and D_h at will; the question is whether they can produce σ_h. Lemma 4.8 establishes that producing such a σ_h without access to sk_i requires breaking the existential unforgeability of ed25519, which is infeasible under the discrete log assumption on the curve25519 group [3]. □

Lemma 4.7 (Attribution Uniqueness)**.** *A verifying APB uniquely identifies a registered principal: there is no efficient way to produce an APB whose σ_h verifies under two distinct registered pk_i simultaneously.*

Proof. Two distinct ed25519 public keys verify the same signature over the same message only with negligible probability (the signature is bound to the secret key, and finding a colliding key pair reduces to solving discrete log on the curve). □

Lemma 4.8 (Forge Resistance under Computational Hardness)**.** *Any polynomial-time algorithm that produces a verifying σ_h over a chosen message without access to sk_i has success probability negligible in the security parameter.*

Proof. This is the standard EUF-CMA property of ed25519 over the curve25519 group, established under the discrete-log assumption [3, 4]. □

4.4 T8.4 — Finite-Time APB Construction Termination

Theorem 4.9 (Finite-Time APB Construction Termination)**.** *For any persistent halt event h on a system with bounded admission snapshot $\mathcal{A}_0$ and bounded trace $\text{trace}_{\leq t_e}$, the construction of E_s terminates in time bounded by $O(|\mathcal{A}_0| + |\text{trace}_{\leq t_e}|)$.*

Proof. By Definition 3.3, E_s has five fields: $\text{hash}(\mathcal{A}_0)$, $\widehat{D}(t_e)$, t_e, $\text{hash}(\text{trace}_{\leq t_e})$, and cause. Lemma 4.10 establishes that the field count is fixed; Lemma 4.11 establishes that each field's capture time is bounded. Their conjunction gives the stated bound. □

Lemma 4.10 (Bounded E_s Field Count)**.** *The number of fields in E_s is fixed at five and does not depend on runtime state.*

Proof. By Definition 3.3, fixed at compile time. □

Table 2: Implementation modules introduced in this work. LOC counts include docstrings.

Module	LOC	Responsibility
`agent/principal.py`	140	Principal Set $\mathcal{P}$, keypair generation, registry, revocation.
`stack/apb.py`	200	E_s, D_h, APB classes; canonical encoding; signing.
`stack/apb_verifier.py`	140	4-predicate verification; failure-mode enum; attribution.
`stack/governance_layer.py`	190	Authority Resolution Function G; built-in policies.
Total	670	Plus 61 unit tests over the four modules.

Lemma 4.11 (Bounded Capture Time Per Field)**.** *Each of the five E_s fields can be computed in time bounded linearly in the size of its underlying input:* $\mathrm{hash}(\mathcal{A}_0)$ *in* $O(|\mathcal{A}_0|)$*;* $\widehat{D}(t_e)$ *in* $O(1)$ *(already computed by the IML monitor);* t_e *in* $O(1)$*;* $\mathrm{hash}(\mathrm{trace}_{\leq t_e})$ *in* $O(|\mathrm{trace}_{\leq t_e}|)$*;* cause *in* $O(1)$ *(a fixed-vocabulary string).*

Proof. SHA-256 is linear in input size; the remaining operations are elementary. □

5 Implementation

We provide a real-cryptography reference implementation of the APB in Python on top of the existing governance stack of [7]. All four theorems' empirical claims (§6–§9) are verified against this implementation.

Cryptography. We use ed25519 from the `cryptography` library [1], version ≥ 42. Each $H_i \in \mathcal{P}$ has a 32-byte private key (held only by H_i) and a 32-byte public key held in the registry. Signatures are 64 bytes. Canonical serialisation uses Python's `json.dumps` with `sort_keys=True` and the compact separators `(",", ":")`.

Modules. Four new modules implement the framework (see Table 2).

Verifier predicates. The verifier evaluates four predicates against an APB and a registry, and returns the first failing predicate (or VALID):

V1. The signed message verifies against pk_i for the H_i named in D_h.

V2. H_i is in the registry.

V3. H_i was active at t_e (not revoked before signing).

V4. t_e lies within an acceptable window relative to the verification time (replay defence).

The enumerated failure modes (`INVALID_SIGNATURE`, `PRINCIPAL_NOT_FOUND`, `PRINCIPAL_REVOKED`, `REPLAY`, `MALFORMED`) allow Experiment B (§7) to count detection rates per attack class.

Persistence. APBs are persisted in append-only JSONL files with HMAC chaining: each entry's HMAC is computed over the previous entry's HMAC concatenated with the current entry's canonical bytes. This gives log-level tamper evidence orthogonal to the per-APB signature: any deletion

or reordering of entries breaks the chain. We do not use a Merkle tree in this implementation (it adds machinery for a distributed-log scenario that the present paper does not need); we note in §10 that a Merkle structure is the natural extension for distributed deployments.

Decision policies. We provide three built-in `DecisionPolicy` callables that the governance layer can route to: `always_resume`, `always_deny`, and `threshold_policy(deny_above, recalibrate_above)`. The threshold policy issues `RESUME` when $\widehat{D}(t_e) <$ `deny_above`; `DENY` when `deny_above` $\leq \widehat{D}(t_e) <$ `recalibrate_above`; `RECALIBRATE` otherwise. The choice of (θ_d, θ_r) is operator configuration; Experiment A (§6) shows that governance completeness is invariant under this choice.

5.1 Proposition 5.1 — T^* Calibration

Proposition 5.1 (T^* Calibration Range)**.** *Let model M have measured drift threshold T^*_M with run-to-run standard deviation σ_M. The persistence window ΔT in DC.2 satisfies the calibration criterion if*

$$k_1 \cdot \sigma_M \;\leq\; \Delta T \;\leq\; k_2 \cdot T^*_M,$$

with $k_1 \geq 3$ to bound the false-positive rate below the run-to-run noise of T^* and $k_2 \leq 0.5$ so that the persistence window declares a halt before the measured drift threshold is reached.

This is a design recipe rather than a deductive theorem: k_1 and k_2 are operator choices reflecting tolerance to false positives and false negatives respectively. The empirical content of Proposition 5.1 is the claim that the range $[k_1\sigma_M, k_2 T^*_M]$ is non-empty for the models we measure — equivalently, that σ_M / T^*_M is small. We verify this in §8 for five of the six models in our study; one model does not drift within the experimental window and is treated separately in §10.

6 Experiment A: Governance Completeness

Goal. Empirical demonstration that T8.1 holds: every runtime halt produced by the governance stack resolves through either the recovery loop (R1) or a signed APB (R2), with no third path observed.

Setup. The full stack ACP + IML + RAM + RecoveryLoop + GovernanceLayer is run for 1000 steps per seed (50-step burn-in plus 950 drift steps), with 10 seeds (`42`–`51`), coverage 0.70, $\theta = 0.20$, and a single principal H_{alice} in $\mathcal{P}$. Whenever the recovery loop returns `HALT` or `DENY` on a halt, the governance layer is invoked with a threshold-based decision policy; the resulting APB is verified through the verifier (V1–V4) before being recorded.

Policy sensitivity. We run the same stack under two threshold policies. The *default* policy uses $(\theta_d, \theta_r) = (0.40, 0.70)$, which causes essentially all governance events to receive `RESUME` given the empirical $\widehat{D}$ distribution under this protocol. The *low-threshold* policy uses $(0.25, 0.32)$, which exercises the full $\{$`RESUME`, `DENY`, `RECALIBRATE`$\}$ output range. Both policies are run against the same seeds.

Table 3: Governance Completeness (T8.1) (default policy): every HALT resolves through Recovery or a signed APB. NEITHER count is 0 across all seeds (assertion PASSED).

Seed	HALTs	Recovery RESUME	APB-signed	NEITHER
42	396	211	185	0
43	401	207	194	0
44	368	174	194	0
45	387	191	196	0
46	364	201	163	0
47	387	217	170	0
48	371	168	203	0
49	378	173	205	0
50	398	231	167	0
51	362	239	123	0
Total	**3812**	**2012**	**1800**	**0**

Table 4: Policy sensitivity in Exp A: T8.1 (Governance Completeness) holds identically under both threshold policies. The APB decision distribution varies with policy parameters; Completeness is policy-agnostic.

Policy	(θ_d, θ_r)	HALTs	Rec. RESUME	APB-signed	RES/DENY/RECAL	NEITHER
default	(0.40,0.70)	3812	2012	1800	1800/0/0	0
low_thresh	(0.25,0.32)	3812	2012	1800	1037/727/36	0

Result. Table 3 reports the per-seed and aggregate counts for the default policy. Across 10 seeds and 3,812 halt events, every halt resolves through exactly one of the two paths; no third path is observed. The *neither* count, which would witness a violation of T8.1, is zero. Table 4 shows that the low-threshold policy produces an identical halt count and an identical APB count, but a different distribution of decisions (1037 `RESUME` / 727 `DENY` / 36 `RECALIBRATE` instead of 1800/0/0): completeness is policy-invariant; the decision distribution is not.

Reading. The two tables together separate two distinct empirical claims. T8.1 is about *path exhaustivity*: every halt ends through R1 or R2, never neither, regardless of operator policy. The decision distribution within R2 is a property of the policy, not of T8.1; the framework is policy-agnostic.

7 Experiment B: APB Integrity

Goal. Empirical demonstration that T8.2 and T8.3 hold: no modification of an APB after signing is undetectable, and no forgery without sk_i verifies. We exercise nine adversarial vectors covering the threat-model classes A1–A4 of §3.5.

Setup. 200 freshly-signed APBs are generated under varied evidence (each E_s uses random A_0-hash, varying $\widehat{D}$ across $[0.20, 0.45]$, and a current t_e). A baseline run confirms all 200 untampered

Table 5: APB Integrity (T8.2 + T8.3): four attack vectors against 200 freshly-signed APBs. All tampering attempts are detected by the verifier. T8.2: PASSED. T8.3: PASSED.

Attack vector	Trials	Detected	Missed	Detection rate
Tamper E_s.A_0_hash	200	200	0	100.00%
Tamper E_s.D_hat	200	200	0	100.00%
Tamper E_s.t_e	200	200	0	100.00%
Tamper E_s.trace_hash	200	200	0	100.00%
Tamper E_s.cause	200	200	0	100.00%
Forge σ_h (random bytes)	200	200	0	100.00%
Forge σ_h (attacker key)	200	200	0	100.00%
Identity swap (H_i replaced)	200	200	0	100.00%
Replay (at $t_e + 360\,\mathrm{s}$)	200	200	0	100.00%
Total	**1800**	**1800**	**0**	**100.00%**

APBs verify cleanly. Each attack vector is then applied to each APB, and the verifier output is recorded.

Vectors.

A1.1–A1.5: tamper E_s field-by-field. For each of the five E_s fields (`A_0_hash`, `D_hat`, `t_e`, `trace_hash`, `cause`), substitute a modified value while retaining D_h and σ_h.

A2a: forge with random bytes. Replace σ_h with 64 random bytes.

A2b: forge with attacker keypair. Re-sign $\mathrm{canon}(E_s) \,\|\, \mathrm{canon}(D_h)$ with a freshly-generated key whose public counterpart is not in $\mathcal{P}$.

A3: identity swap. Replace H_i in D_h with a different registered principal, retaining σ_h.

A4: replay. Verify the (untampered) APB at a clock value $t_e + 360\,\mathrm{s}$ with replay window $300\,\mathrm{s}$.

Result. Table 5 reports the per-vector counts. Across 1,800 attack attempts, the verifier detects every attack: 1800/1800, 0 misses, 100.0000% detection rate. T8.2 and T8.3 hold empirically.

Reading. The five E_s-tampering vectors collectively exercise Lemma 4.5 on every E_s field, witnessing T8.2. The two forgery vectors witness T8.3 (the system layer cannot produce a valid signature without sk_i). The identity-swap vector witnesses Lemma 4.7: substituting H_i in D_h without recomputing σ_h under the new principal's key is detected. The replay vector witnesses the verifier's V4 predicate.

8 Experiment C: Cross-Model T^*

Goal. Empirical characterisation of the model-specific drift threshold T^* across six open LLM families and validation of the calibration criterion of Proposition 5.1.

Table 6: Cross-model T^*: 6 LLM families, 3 runs each, 500 drift steps, $T = 0.4$, $\theta = 0.2$. T^* as mean ± std across runs; σ/T^* is the intra-model coefficient of variation. Proposition 5.1 calibration criterion ($\sigma/T^* < 5\%$ for measured T^*): HOLDS. 1 model(s) did not reach θ within the 500-step window.

Model	Family	Params	T^* (mean±std)	D_{final}	σ/T^*
llama3.2:3b	Meta	3.2B	151 ± 0.9	0.430 ± 0.000	0.62%
gemma4:latest	Google	4.0B	264 ± 5.2	0.272 ± 0.010	1.99%
mistral:7b	Mistral	7.2B	157 ± 0.0	0.434 ± 0.001	0.00%
qwen2.5:7b	Alibaba	7.6B	154 ± 2.9	0.427 ± 0.001	1.91%
deepseek-r1:8b	DeepSeek	8.0B	160 ± 0.8	0.424 ± 0.000	0.51%
gpt-oss:20b	OpenAI	20.0B	—	0.071 ± 0.005	—

Setup. Six models (`mistral:7b`, `deepseek-r1:8b`, `gemma4:latest`, `gpt-oss:20b`, `qwen2.5:7b`, `llama3.2:3b`) are run via Ollama through the LiveLLM tool selector, 50-step burn-in plus 500 drift steps, sampling temperature $T = 0.4$, $\theta = 0.20$, 3 runs per model. T^* is the first drift-step index at which $\widehat{D} \geq \theta$.

Result. Table 6 reports per-model T^* and final $\widehat{D}$ values, sorted by parameter count. The intra-model coefficient of variation σ/T^* is reported in the rightmost column.

Reading. Five of six models reach θ within the 500-step window, with σ/T^* between 0.00% (perfect replication on `mistral:7b`) and 1.99% (`gemma4`). For these five models, the calibration criterion $\sigma/T^* < 5\%$ of Proposition 5.1 holds with substantial margin: the range $[k_1\sigma_M, k_2T^*_M] = [3\sigma_M, 0.5T^*_M]$ is non-empty for every measured model.

The sixth model, `gpt-oss:20b`, the largest in our sample at 20B parameters, does not drift within the experimental window: final $\widehat{D} = 0.071 \pm 0.005$, well below $\theta = 0.20$. We discuss this case explicitly in §10; for the purposes of T8.1's empirical verification it is sufficient to note that the calibration criterion is reported per-model rather than as a global property, and that the deployment recipe of Proposition 5.1 requires a measured T^* before ΔT can be set.

Family-wise consistency. The Mistral family entry in our sample (`mistral:7b`, $T^* = 157 \pm 0$) coincides with the Mistral-family T^* reported in [7] for a larger `mistral-small-3.1` model under the same protocol ($T^* = 156 \pm 1$). Two distinct sizes within the same family yield T^* within one step, supporting the architectural rather than scale-driven character of the threshold.

Refutation of size-monotonicity. The largest model in the sample is the only one that does not drift. This refutes any monotone hypothesis under which larger models would be more (or less) susceptible to compliant drift than smaller ones. The empirical implication is that T^* must be *measured* per deployment rather than *predicted* from architectural parameters.

Table 7: Temperature insensitivity of T^*. Three models were each run at four sampling temperatures, three runs per cell, 500 drift steps. Cell entries are mean ± std of T^* across runs at that temperature; the rightmost column is the per-model range $\max_T T^*(T) - \min_T T^*(T)$ of the cell means. Insensitivity criterion (range ≤ 10 steps): VIOLATED.

Model	$T = 0.2$	$T = 0.4$	$T = 0.6$	$T = 0.8$	range
gemma4:latest	239±6.9	255±7.3	249±3.9	290±18.4	51
llama3.2:3b	155±0.9	155±1.6	156±1.2	156±2.5	2
mistral:7b	154±2.6	157±2.2	156±1.9	152±1.6	5

9 Experiment D: Temperature Insensitivity

Goal. Test the conjecture that T^* is determined by the escalation protocol embedded in the system prompt rather than by LLM sampling stochasticity. Empirical translation: across a sweep of sampling temperatures, the per-model spread of mean T^* should be small.

Setup. Three models from §8 are selected for diversity in baseline T^*: `mistral:7b` ($T^* = 157$, "fast cluster"), `llama3.2:3b` ($T^* = 151$, "fastest"), and `gemma4:latest` ($T^* = 264$, "slowest drifter"). Each is run at temperatures $T \in \{0.2, 0.4, 0.6, 0.8\}$, three runs per cell, 500 drift steps, identical burn-in and θ to §8. The insensitivity criterion is that the per-model range $\max_T T^*(T) - \min_T T^*(T)$ across the four temperatures is at most 10 steps.

Result. Table 7 reports the per-cell T^* across the four temperatures and the resulting per-model range. The result is more nuanced than a single insensitivity claim: two of three models satisfy the criterion with substantial margin, and the third violates it.

Reading. For the two fast-drifting models, `llama3.2:3b` and `mistral:7b`, T^* is strongly temperature-insensitive: the range across $T \in \{0.2, 0.4, 0.6, 0.8\}$ is 2 and 5 steps respectively, and the within-cell standard deviations are below 3 steps for every $(model, T)$ pair. For these models the conjecture of [7] — that T^* is determined by the escalation protocol embedded in the system prompt rather than by sampling stochasticity — replicates with substantial margin and generalises across families.

For `gemma4`, the slow-drifting model in our sample, the picture is different. The per-cell mean at $T = 0.8$ rises to 290 ± 18.4 steps, against 239–255 at the lower temperatures, giving a per-model range of 51 steps and a within-cell standard deviation that is roughly three times larger than at $T \leq 0.6$. The insensitivity criterion (range ≤ 10) is violated.

Interpretation. We read this as evidence for a *drift-floor* effect: when a model's T^* is close to the lower bound that the escalation protocol can produce, sampling temperature has little room to move the threshold. When T^* is well above that floor (as in `gemma4`, whose T^* is 1.7× larger than the fast-cluster), the additional steps before threshold crossing admit cumulative sampling fluctuations that compound at higher temperatures. The relevant quantity is therefore not whether temperature insensitivity holds globally but whether a model's baseline T^* sits in the deterministic

regime; this is itself a measurable property of the deployment and feeds back into the calibration recipe of Proposition 5.1 (a model in the non-deterministic regime requires a wider $k_1\sigma_M$ margin to absorb temperature-induced spread).

Implication for governance. Operators deploying models in the slow-drifting regime should not assume that a single-temperature characterisation of T^* generalises to other temperatures of the same model. Either the deployment temperature is fixed (and T^* measured at that temperature suffices), or the calibration is performed at the operationally relevant temperature(s). For fast-drifting models, a single-temperature characterisation appears to be sufficient.

10 Discussion

Models that do not drift. The case of `gpt-oss:20b` in §8 merits explicit treatment. Under our 500-step protocol the model's drift estimator remains well below threshold; we cannot, from this data alone, distinguish three explanations: (i) the model is architecturally robust to the specific escalation protocol embedded in the system prompt and would not drift at any reasonable horizon, (ii) the model has a genuinely larger T^* that exceeds our window, (iii) the model's tool-selection distribution under our LiveLLM prompt is structurally different from the other five (e.g. a strong default toward low-risk actions that does not respond to context-pressure scaling). For deployment, this distinction matters: under (i) the model would not need an APB-emitting governance layer at all under this protocol, while under (ii) or (iii) the operator would extend the window or adapt the prompt. In either case, the implication for governance design is the same: $\mathcal{P}$ and G are configured once; ΔT is configured per deployment from a measured T^*, with the explicit understanding that T^* may be infinite or undefined for some models.

Boundaries of the result. The APB establishes accountability, not access control. A compromised principal can sign a malicious decision and the APB will record it. The mitigations are configurable around the APB, not within it: k-of-n multi-signature for sensitive decisions, key rotation under `RECALIBRATE` governance events, and out-of-band review of recorded APBs against expected decision patterns. The proofs of T8.1–T8.4 rest on the security assumptions of ed25519 over curve25519. A break of those assumptions would invalidate the cryptographic guarantees but not the structural claims (T8.1 in particular is a partition argument that does not depend on the signature scheme).

Integration paths. Two integration directions are natural extensions and deferred to subsequent work. *MCP/A2A integration*: the governance stack of [7] operates on tool-call traces from a LangGraph state graph; the Model Context Protocol and Agent-to-Agent communication standards define interception points at which the APB-emitting layer could be applied transparently to agent code. *Distributed APB log*: replacing the JSONL+HMAC chain with a Merkle tree (and, for cross-organisational deployments, anchoring to a public transparency log) gives stronger tamper evidence under multi-writer scenarios.

Choice of decision-policy thresholds. Experiment A's policy comparison shows that Governance Completeness is invariant under threshold choice. We do not, in this paper, prescribe a

particular (θ_d, θ_r). Their setting is a deployment decision tied to the operator's tolerance for halts that escalate to `DENY` versus `RESUME`, and to the empirical distribution of $\widehat{D}$ at the moments of governance event in the operator's workload.

11 Conclusion

We have introduced the Accountability Proof Block as the formal mechanism by which an LLM agent's persistent halts resolve into identity-bound, cryptographically verifiable governance decisions. The construction is supported by four theorems — governance completeness, non-repudiability, impossibility of anonymous re-authorisation, and finite-time evidence construction — and a real-cryptography reference implementation. Empirical validation across 3,812 halt events, 1,800 adversarial attempts, 18 cross-model runs, and 36 temperature-sweep runs shows that the construction behaves as the theory predicts and that the model-specific drift threshold T^* is stable, measurable, and architecture-driven rather than scale-driven, with the empirical caveat that temperature-insensitivity itself is conditional on a model sitting in a fast-drifting regime. The result is a small, configurable, provably-bounded mechanism by which a system that has reached the limit of its own authority can hand the resolution back to a named, accountable human — without trusting the system itself to make that handover honestly.

References

[1] Python Cryptographic Authority. cryptography: Python cryptographic recipes and primitives. https://github.com/pyca/cryptography, 2024. Version 42+.

[2] Yuntao Bai et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

[3] Daniel J. Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. High-speed high-security signatures. *Journal of Cryptographic Engineering*, 2(2):77–89, 2012. ed25519 specification.

[4] Dan Boneh and Victor Shoup. *A Graduate Course in Applied Cryptography*. Self-published, 2014. URL https://crypto.stanford.edu/~dabo/cryptobook/.

[5] Dan Boneh, Manu Drijvers, and Gregory Neven. Compact multi-signatures for smaller blockchains. In *ASIACRYPT 2018*, pages 435–464, 2018.

[6] Tim Bray. The JavaScript Object Notation (JSON) data interchange format. RFC 8259, 2017.

[7] Marcelo Fernandez. Closing the execution gap in LLM agent systems: Empirical evidence for compliant drift, partial observability, and integrated runtime governance. arXiv:TBD, 2026. Paper 7 of the Agent Governance Series. https://doi.org/10.5281/zenodo.19929771.

[8] Rosario Gennaro and Steven Goldfeder. Fast multiparty threshold ECDSA with fast trustless setup. In *ACM CCS 2018*, pages 1179–1194, 2018.

[9] Ben Laurie, Adam Langley, and Emilia Käsper. Certificate transparency. RFC 6962, 2013.

[10] Margaret Mitchell, Simone Wu, Andrew Zaldivar, Parker Barnes, Lucy Vasserman, Ben Hutchinson, Elena Spitzer, Inioluwa Deborah Raji, and Timnit Gebru. Model cards for model reporting. In *Proceedings of the Conference on Fairness, Accountability, and Transparency (FAT*)*, pages 220–229, 2019.

[11] OpenAI. GPT-4 system card. https://openai.com/index/gpt-4-system-card/, 2024. Accessed 2026-05-08.

[12] Gavin Wood. Ethereum: A secure decentralised generalised transaction ledger. Ethereum Yellow Paper, 2014. EIP-150 Revision.